J-Reflect Recovery: development-split analysis

Qwen3.5-9B with the released J++ lens · Modal run on an NVIDIA L40S, 2026-11-03 · dev split only (test split not run) · cost ≈ $0.53

Bottom line

The run executed the protocol correctly, but this configuration cannot test the hypothesis. The model answered **0/200** problems correctly, **0** problems were eligible for the latent-detection measure, and every feedback condition corrected **0%** of errors. That is not evidence against the idea that decoded internal information can help. The analysis below finds four independent design blockers, any one of which would force a null result.

Key findings

- **Task too hard for instant answers.** 0/200 correct, including 4 within 1% of the right answer. Answers are usually the right order of magnitude (135/200 have the correct number of digits), consistent with estimating rather than computing.
- **The detection measure cannot apply to this model.** Qwen3.5-9B's 248,044-token vocabulary has no multi-digit number tokens; every number is split into single digits. All 200 hidden intermediates (e.g. 84) were ineligible, so detection, the freeze gate, and the ablations had nothing to measure.
- **Second attempts copy the first.** 799/800 second answers repeat the first answer exactly, including 199/200 after the model is told its answer was wrong. Under greedy, answer-only decoding, feedback has no observable effect, so the correction comparison sits at a floor.
- **Readouts describe the task type, not the problem.** Each layer's J++ top-10 uses only 11–19 distinct tokens across all 200 problems. A problem's own readout differs from its shuffled donor's by 4.8 of 70 tokens on average, so the authentic and shuffled prompts are nearly the same text.
- **No instance-specific numeric signal.** The first digit of the hidden intermediate appears in the problem's own readout 117/200 times and in a different problem's readout 115/200 times.
- **J++ is qualitatively more interpretable than the logit lens:** its layers progress from formatting tokens to math/arithmetic concepts to “calculation” to “Answer”. The logit lens on the same activations is mostly unrelated multilingual fragments.

Recommended next step

Redesign before spending more. Use single-token (word-valued) intermediates, calibrate difficulty so first-attempt accuracy is roughly 30–80%, give every second attempt the same small reasoning budget so feedback can change the answer, and add cheap pre-flight checks on a small dev sample (eligibility, accuracy, readout specificity, change rate) before running correction arms. Details are in Section 6.

1. What was run

The pre-registered protocol (docs/protocol.md) asks whether the J++ readout of a frozen model's own activations, given back to it as feedback, helps it fix wrong answers more than control feedback does. Each problem has a known hidden intermediate value. The model answers once; activations are read at the last prompt token before it answers; then four independent second attempts are generated:

Condition	Feedback after a wrong first answer
retry_only	Neutral: review your answer and correct it if needed
error_only	Your answer was incorrect
shuffled_jpp	Incorrect + J++ readout from a different problem (same template)
authentic_jpp	Incorrect + J++ readout from this problem's own activations

Run configuration

Setting	Value
Model	Qwen/Qwen3.5-9B @ c20223623576…, bfloat16, greedy, thinking disabled, ≤64 new tokens
Lens	koayon/jpp-lenses / qwen3.5-9b/lens.pt @ 237dab6ea0e6…; layers [4, 8, 12, 16, 20, 24, 28]; top-10
Capture	last prompt token generation prefill
Data	200 dev problems (3 templates), tail_steps = 3; seed 42
Hardware / environment	NVIDIA L40S on Modal; Python 3.12.11, torch 2.15.0, transformers 5.22.1
Stages completed	prepare → collect (200) → correct (800) → ablate (0 eligible) → report → freeze (gate failed, test not run)
Compute	1000 generations; ≈14 min billed; ≈$0.53

2. Was the run valid?

Yes. Every check below was recomputed from the saved records after the run.

Check	Result
Code that ran equals the repository code	Source hash matches
Installed packages equal uv.lock	All match (installed with uv sync --frozen; identical versions)
Research mode, real GPU	mode = research, device cuda:0
Unit tests and lint	24/24 pass; ruff clean (run locally after the job)
Gold answers consistent with worked steps	200/200
Independent regrade of first answers	0/200 correct (matches the report)
Answers well formed	200 parsed, 0 format failures, all ended normally (none truncated), no thinking text
Readouts captured	All 7 layers for all 200 problems
Correction prompts	800 built; every authentic prompt contains its own readout; every shuffled donor is a different problem (200/200 same template, by design)

Procedural deviations: the automation skipped the README's inspection step (collect 10, report, look at detection) before running corrections, and the Modal image installed with --frozen instead of --locked. Neither affects any number in this report; the first one cost about $0.30 of correction runs that could not have produced a result.

3. Results

3.1 First attempts: 0 of 200 correct

With tail_steps = 3, each question chains five or six operations, including at least two multiplications, and correct answers mostly run to 4–6 digits. Example: *Start with 35. Add 11, then multiply the result by 8. Next multiply the result by 18. Next add 18. Next subtract 15. What is the final number?* The worked steps are 35 + 11 = 46; 46 * 8 = 368; 368 * 18 = 6624; 6624 + 18 = 6642; 6642 - 15 = 6627. The model answered **Answer: 4686**.

Template	n	Correct	Median relative error	Right digit count	Right first digit	Within 1%
add, then multiply	67	0	76%	52	14	1
multiply, then subtract	67	0	78%	32	11	0
variable tracking	66	0	12%	51	46	3
All	200	0	58%	135	71	4

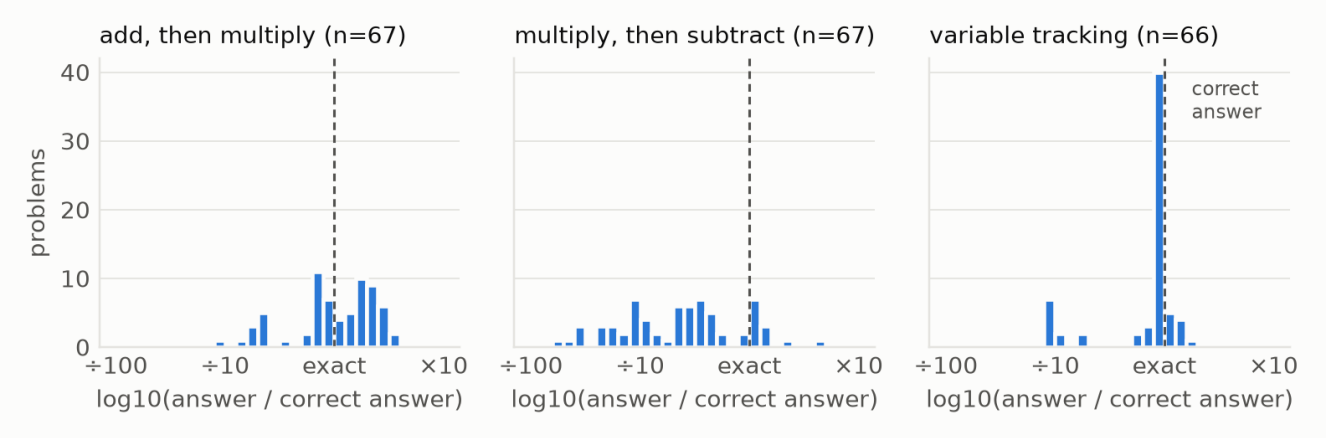


Figure 1. How far off the first answers were, by template. Each bar counts problems; 0 on the horizontal axis is the correct answer, –1 is ten times too small. Answers cluster near the right magnitude but none is exact. Variable-tracking answers are closest (median relative error 12%).

Interpretation: with thinking disabled and an instruction to output only the answer, the model has to produce the result in a single forward pass per token. Its answers have the right shape (digit count, leading digit about a third of the time) but the arithmetic is not carried out. None of the wrong answers equals an intermediate value from the worked steps, so they are not partial computations. The protocol anticipated too few errors; here the problem is the opposite, and with no correct answers the harm analysis (damage to initially correct answers) is also empty.

3.2 No problem was eligible for the detection measure

The detection measure asks whether the hidden intermediate, as one vocabulary token, appears in the readout's top-10 at any layer. That requires the number to be a single token. The tokenizer settles this:

Tokenizer fact (Qwen3.5-9B, pinned revision)	Value
Vocabulary size	248,044
Tokens made only of digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ², ³, ¹
Longest all-digit token	1 character
Pre-tokenizer number rule	\p{N} (each digit is split off individually)
Ineligibility reasons (200 problems)	target_not_single_token: 200

Every hidden intermediate in the dataset has 2–3 digits (the generator requires 10–999), so none can ever be eligible on this model. As a consequence the detection table has N = 0, the freeze gate (≥ 10 eligible errors and J++

above both baselines) failed, and no ablations ran. The repository's tests used tiny random models with a different tokenizer, which is why this was not caught earlier.

3.3 Second attempts repeat the first answer

Correction rate is the pre-registered outcome, but a more sensitive question is whether the second answer moves at all. It almost never does:

Condition	Same answer as first	Closer to correct	Farther	Correct	Prompt tokens	Answer tokens
retry_only	200/200	0	0	0/200	123	10.6
error_only	199/200	1	0	0/200	120	10.6
shuffled_jpp	200/200	0	0	0/200	428	10.6
authentic_jpp	200/200	0	0	0/200	428	10.6

799 of 800 second attempts reproduce the first answer token for token. The only change is one error_only answer, which moved slightly closer but was still wrong. The J++ conditions add about 300 prompt tokens of readout and change nothing. Because the first answer is in the conversation history, greedy decoding with an answer-only instruction simply copies it; being told “incorrect” does not trigger recomputation. Both primary contrasts therefore have zero discordant pairs (difference 0.0 points, interval 0 to 0, Holm p = 1.0). This blocker is independent of the tokenizer: even with eligible targets and perfect readouts, this second-attempt setup would hide any benefit.

3.4 Readouts describe the task type, not the individual problem

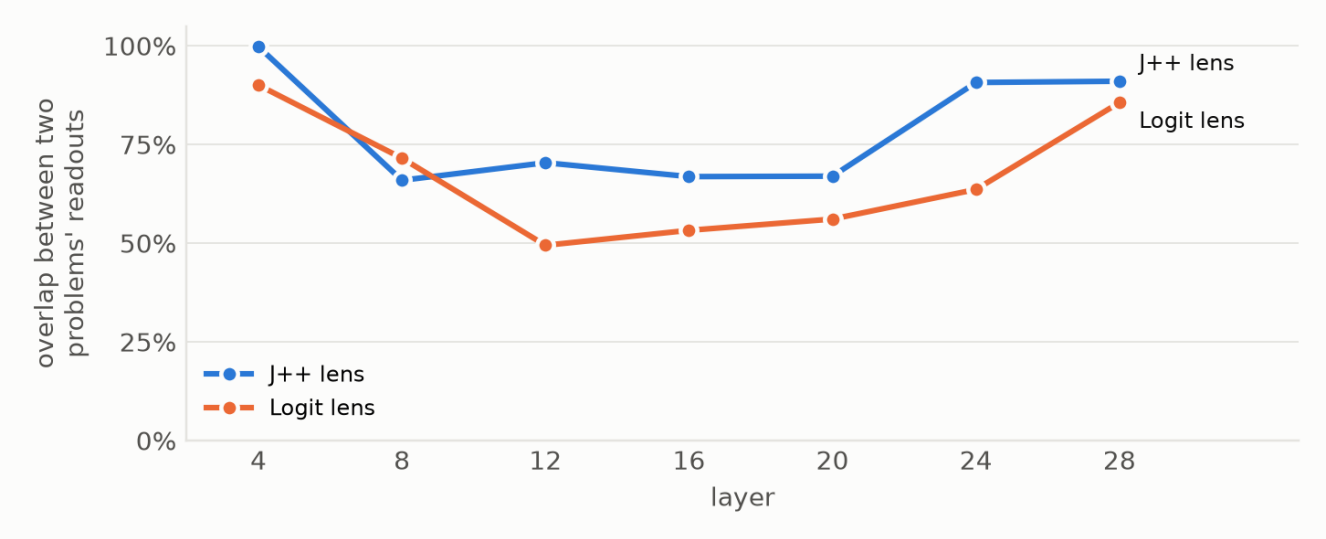


Figure 2. Average overlap (Jaccard similarity) between the top-10 readouts of two different problems at each layer, over 4,000 random pairs. 100% means identical readouts. J++ readouts are 66–100% shared between unrelated problems at every layer.

Layer	Distinct tokens over 200 problems	Most frequent J++ tokens (problems containing it, of 200)
4	11	‘<span’ 200, ‘\n’ 200, ‘<b’ 200, ‘\u’ 200, ‘<strong’ 200
8	19	‘Math’ 200, ‘数学’ 200, ‘ mathematical’ 200, ‘ithmetic’ 193, ‘acers’ 192
12	17	‘2’ 200, ‘’S’ 200, ‘1’ 200, ‘7’ 200, ‘’T’ 200
16	19	‘ Calculation’ 200, ‘ Calculator’ 200, ‘ Arithmetic’ 200, ‘Calc’ 200, ‘Bitcoin’ 199
20	19	‘ calculation’ 200, ‘ Calculation’ 200, ‘计算’ 200, ‘ calculations’ 199, ‘的计算’ 198
24	12	‘@include’ 200, ‘@if’ 200, ‘@g’ 200, ‘@m’ 200, ‘@d’ 200

Layer	Distinct tokens over 200 problems	Most frequent J++ tokens (problems containing it, of 200)
28	11	‘ Answer’ 200, ‘ ANSW’ 200, ‘ answers’ 200, ‘Answer’ 200, ‘_answer’ 200

The J++ readouts form a clear progression: formatting tokens (layer 4), mathematics (8), digits (12), arithmetic and calculation (16–20), code-like fragments (24), and “Answer” (28). They are far more interpretable than the logit lens, whose middle layers are unrelated multilingual fragments. But they are almost the same for every problem. Averaged over problems, the problem's own readout and its shuffled donor's differ by only 4.8 of 70 token slots (donors come from the same template by design). The authentic and shuffled prompts are therefore nearly identical text, so the authentic-vs-shuffled contrast could not separate them even if the model did use feedback.

3.5 No sign of the hidden value in the digits

Layer 12 is the only layer where digits appear (half of its top-10). The same few digits dominate: 2: 200, 1: 200, 7: 200, 9: 197, 3: 150, 0: 54, 8: 4 (problems containing each, of 200). To test whether they carry problem-specific information, each digit 0–9 was classified by where it occurs for that problem:

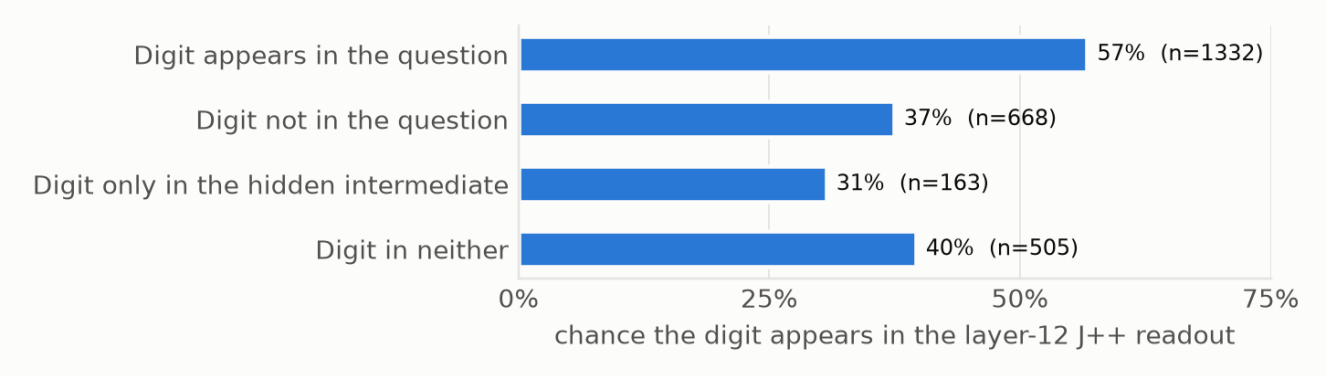


Figure 3. Chance that a digit appears in the layer-12 J++ readout, by where the digit occurs for that problem (2,000 digit-problem pairs). Digits in the question are somewhat more likely to appear; digits found only in the hidden intermediate are slightly less likely than baseline.

The readout leans toward digits that are visible in the question (a copying signal), not toward digits of the hidden intermediate. The first digit of the hidden intermediate appears in the problem's own J++ readout 117/200 times and in a different problem's readout 115/200 times, which is no difference. These are exploratory, post hoc checks, not part of the pre-registered analysis.

3.6 Where the activations were read

The protocol captures the last prompt token during generation prefill. With thinking disabled, Qwen's chat template inserts an empty thinking block, so the last ten prompt tokens are ‘?’, ‘<|im_end|>’, ‘\n’, ‘<|im_start|>’, ‘assistant’, ‘\n’, ‘<think>’, ‘\n\n’, ‘</think>’, ‘\n\n’ and the captured position is the blank line after `</think>`. The model's next output is ‘Answer’, followed by the digits one at a time (‘Answer’, ‘:’, ‘·’, ‘1’, ‘3’, ‘0’, ‘8’ …). This explains why the last layers read “Answer”: at this position the model is predicting the word “Answer”, nine tokens after the question’s final “?”. Whether the intermediate is represented earlier (for example at the question's final tokens) or while the answer digits are generated was not tested. The README already notes that a prefill snapshot may precede the relevant computation.

4. What these results do and do not show

They show:

- The pipeline, capture, lens loading and statistics work end to end on the real 9B model, cheaply (about 0.5 s per answer).

- This configuration (multi-digit numeric targets, hard answer-only arithmetic, greedy answer-only retries, last-prompt-token capture) cannot produce a measurable effect, for four independent reasons.
- At this position, the released J++ lens recovers task-level concepts (mathematics, calculation, answer) much more cleanly than the logit lens.

They do not show:

- That the model lacks the intermediate value internally. With 0 eligible problems, detection was never measured.
- That J++ feedback is useless. The second-attempt setup could not respond to any feedback, including a plain “you were wrong”.
- Anything about the held-out test split, which correctly never ran.

5. The four blockers

Blocker	Evidence	Effect on the experiment
Numeric targets are never single tokens	No multi-digit tokens in a 248,044-token vocabulary; 200/200 ineligible	Detection, freeze gate and ablations have N = 0
Task far too hard	0/200 correct; answers near the right magnitude	No correct answers to protect (harm analysis empty); nothing distinguishes easy from hard errors
Second attempts cannot change	799/800 identical to the first answer, including 199/200 after “incorrect”	Correction rate at 0% in every arm; contrasts have no discordant pairs
Readouts are generic	Own vs donor readouts differ by 4.8/70 tokens; 66–100% overlap across problems	Authentic and shuffled feedback are nearly the same text

6. Recommendations for the next run

Ordered by importance. Each should be fixed on a new development run before any test-split spend.

- **Use single-token intermediates.** Replace numeric intermediates with word-valued ones that are single tokens in Qwen's vocabulary, for example entity or attribute tracking (which colour, which name, which object ends up where) where the hidden step is a word. Check eligibility with the real tokenizer during prepare, not tiny test models. Alternative: switch to a model whose tokenizer keeps multi-digit numbers whole and that has a released lens (OLMo-3-32B may qualify; verify first), at several times the cost.
- **Calibrate difficulty on dev.** Aim for 30–80% first-attempt accuracy so there are enough errors to correct and enough correct answers for the harm analysis. Start from tail_steps = 0 or smaller operands, and choose before preparing the run.
- **Let the second attempt change.** Give every arm the same short reasoning budget (for example enable thinking with a fixed token cap, or allow brief working before the final answer), so a correct hint can actually be used. Keep the budget identical across arms so the information source stays the only difference.
- **Capture at more than one position.** Record readouts at the question's final tokens as well as at the answer start, choose the position on dev only, and freeze it. The current position mostly encodes “about to write Answer”.
- **Make the readout instance-specific.** If raw top-10 lists stay generic, consider a fixed, label-free transformation such as subtracting the template-average readout before taking the top-k, chosen on dev and frozen before test.
- **Add pre-flight checks (a few cents each) before running correction arms:** at least the minimum number of eligible targets; first-attempt accuracy inside the target band; own-vs-donor readout difference well above zero; and, on about 20 wrong answers, error_only changing the answer a meaningful fraction of the time. The automation should stop after collection for inspection, as the README intends.

Appendix A. One complete example

Question	Start with 35. Add 11, then multiply the result by 8. Next multiply the result by 18. Next add 18. Next subtract 15. What is the final number?
Worked steps	35 + 11 = 46; 46 * 8 = 368; 368 * 18 = 6624; 6624 + 18 = 6642; 6642 - 15 = 6627
Correct answer / hidden intermediate	6627 / 46 (ineligible: target_not_single_token)
First answer	Answer: 4686 (wrong)

Readouts at the captured position

Layer	J++ top-10	Logit lens top-10
4	‘<u’, ‘<br’, ‘\u’, ‘<b’, ‘<span’, ‘\n’, ‘<strong’, ‘\s’, ‘<a’, ‘爲’	‘1’, ‘d’, ‘C’, ‘ C’, ‘A’, ‘正’, ‘eng’, ‘一’, ‘x’, ‘ly’
8	‘数学’, ‘Math’, ‘ mathematical’, ‘ Mathematical’, ‘ithmetic’, ‘ Adobe’, ‘ Euler’, ‘acers’, ‘geo’, ‘Calcul’	‘卓’, ‘技’, ‘横’, ‘依法’, ‘math’, ‘inking’, ‘晏’, ‘数学’, ‘ros’, ‘ math’
12	‘’T’, ‘’S’, ‘1’, ‘ Calculation’, ‘2’, ‘’an’, ‘7’, ‘’B’, ‘9’, ‘’n’	‘昔’, ‘牙’, ‘完整的’, ‘法’, ‘蜂’, ‘先行’, ‘ipot’, ‘位’, ‘纠正’, ‘计算’
16	‘ Calculator’, ‘ Calculation’, ‘Bitcoin’, ‘ Arithmetic’, ‘Calc’, ‘CT’, ‘的计算’, ‘算力’, ‘推理’, ‘RM’	‘位数’, ‘概’, ‘理’, ‘ ans’, ‘尾’, ‘ jih’, ‘莱特’, ‘算’, ‘’ve’, ‘的计算’
20	‘计算’, ‘ calculation’, ‘ Calculation’, ‘ calculations’, ‘的计算’, ‘ Carn’, ‘Calc’, ‘ Answer’, ‘@g’, ‘ calculate’	‘TTTT’, ‘三八’, ‘ radios’, ‘OwnerId’, ‘noop’, ‘算’, ‘ ans’, ‘接’, ‘まず’, ‘</think>’
24	‘@c’, ‘@g’, ‘@m’, ‘@s’, ‘@d’, ‘@end’, ‘@a’, ‘@if’, ‘@include’, ‘@Data’	‘ ans’, ‘ Answer’, ‘Answer’, ‘晗’, ‘ansible’, ‘noop’, ‘ans’, ‘mente’, ‘uffle’, ‘engan’
28	‘ Answer’, ‘ answer’, ‘Answer’, ‘ ANSW’, ‘ Answers’, ‘_answer’, ‘ answers’, ‘.answer’, ‘ answering’, ‘答案’	‘ Answer’, ‘Answer’, ‘ answer’, ‘_answer’, ‘答案’, ‘ ANSW’, ‘-answer’, ‘.answer’, ‘answer’, ‘答案是’

Second attempts

Condition	Feedback message	Second answer
retry_only	Review your previous answer. Keep it if correct; otherwise correct it. Return only Answer: <integer>.	Answer: 4686
error_only	Your previous answer was incorrect. Solve the problem again. Return only Answer: <integer>.	Answer: 4686
shuffled_jpp	Your previous answer was incorrect. Solve the problem again. Automatically extracted diagnostic tokens follow. They may be noisy or irrelevant. [{"layer": 4, "tokens": ["<u", "<br", "\\u", "<b", "<span", "\\n", "<strong", "\\s", "<a", "爲"]}, {"layer": 8, "tokens": ["数学", "Math", " mathematical", " Mathematical", "ithmetic", " Adobe", " Euler", "acers", "geo", "Calcul"]}, {"layer": 12, "tokens": ["’T", "’S", "1", "7", … [readout list continues]	Answer: 4686
authentic_jpp	Your previous answer was incorrect. Solve the problem again. Automatically extracted diagnostic tokens follow. They may be noisy or irrelevant. [{"layer": 4, "tokens": ["<u", "<br", "\\u", "<b", "<span", "\\n", "<strong", "\\s", "<a", "爲"]}, {"layer": 8, "tokens": ["数学", "Math", " mathematical", " Mathematical", "ithmetic", " Adobe", " Euler", "acers", "geo", "Calcul"]}, {"layer": 12, "tokens": ["’T", "’S", "1", " Ca … [readout list continues]	Answer: 4686

Appendix B. Methods and files

- Pre-registered numbers (accuracy, eligibility, correction rates, contrasts) come from reports/dev/summary.json and were reproduced independently from the raw records.
- Post hoc analyses (error magnitudes, change rates, readout overlap, digit analysis) were computed from first/dev/*.json, correction/dev/*/*.json, correction/dev/plan.json and the pinned Qwen3.5-9B tokenizer. They are exploratory.

- Readout overlap is Jaccard similarity of top-10 token sets for 4,000 random problem pairs (seed 0). Relative error is |answer – correct| / |correct|.
- Timing: first attempt 0.48 s per generation plus 0.09 s for readouts; 83 prompt and 10.6 answer tokens on average.
- Files: runs/modal/pilot/ (LLM_BUNDLE.md, reports/dev/report.md, summary.json, trials.csv, raw records).